MINISTERUL EDUCAȚIEI NAȚIONALE

UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

Proiect Ingineria Sistemelor

Identificarea sistemelor de ordin II utilizând metoda regresiei liniare

Student: Ghiță Daniel Gulin

Profesor coordonator: Mircea Susca

$$\boxed{\mathbf{m = 6, \quad n = 10}}$$

1 Introducere

Proiectul vizează identificarea modelului matematic pentru un sistem hidraulic de ordinul II utilizand metoda regresiei liniare. Se utilizeaza semnale de tip treapta, iar validarea se realizează prin compararea datelor experimentale cu cele simulate.

2 Calibrarea semnalului de intrare

Această etapă a constat în definirea amplitudinii treptei și a momentului de aplicare a acesteia, folosind variabilele de proces c_1 și c_2.

- **Amplitudinea $u_{st} = 2$:** A fost aleasă pentru a asigura ca semnalul de intrare are o amplitudine corespunzatoare.
- **Timpul $t_1 = 10 \cdot c_2/c_1$:** Acest interval reprezintă o perioadă de așteptare necesară pentru ca sistemul să se stabilizeze la valoarea de pornire.

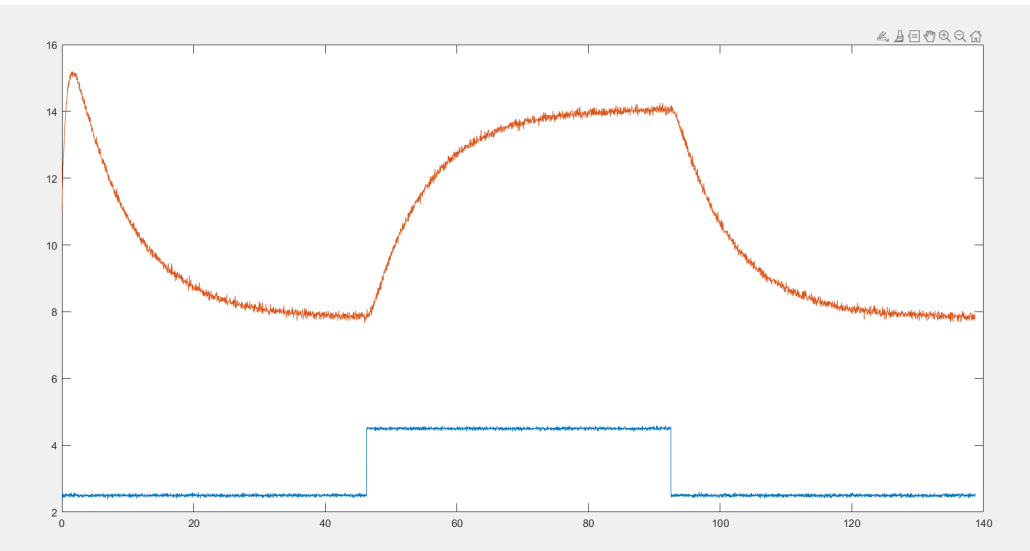

Figura 1: Răspunsul sistemului la semnalul de intrare de tip treaptă

3 Determinarea Factorului de Proporționalitate (K)

Definiție: Factorul de proporționalitate K reprezintă raportul dintre variația mărimii de ieșire și variația mărimii de intrare în regim staționar. Acesta indică amplificarea sistemului, arătând cât de mult se modifică nivelul de ieșire față de modificarea aplicată la intrare.

Pentru a calcula valoarea lui K, am ales patru indici de timp (i_1, i_2, i_3, i_4) care definesc două ferestre în care sistemul este stabil:

- **Regimul inițial:** Indicii $i_1 = 6390$ și $i_2 = 6515$ au fost folosiți pentru a calcula mediile u_0 și y_0.
- **Regimul final:** Indicii $i_3 = 12990$ și $i_4 = 13175$ au fost folosiți pentru a determina valorile u_{st} și y_{st}.

Valoarea factorului de proporționalitate a fost calculată cu formula:

$$K = \frac{y_{st} - y_0}{u_{st} - u_0} \tag{1}$$

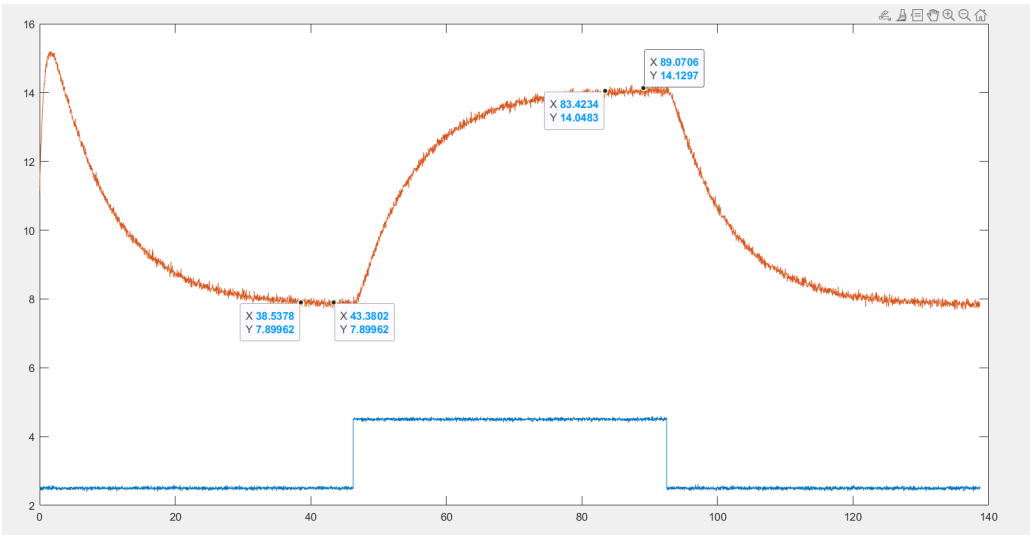

Figura 2: Identificarea indiciilor i_1, i_2, i_3, i_4.

Implementarea în MATLAB: Pentru calculul numeric al valorilor medii și a factorului K, am utilizat următoarea secvență de cod:

```
i1=6390;
i2=6515;
i3=12990;
i4=13175;
u0=mean(u(i1:i2));
ust=mean(u(i3:i4));
y0=mean(y(i1:i2));
yst=mean(y(i3:i4));
K= (yst-y0)/(ust-u0);
```

4 Determinarea constantei de timp dominante (T_1)

Pentru a determina T_1, am utilizat **metoda regresiei liniare**. Deoarece răspunsul sistemului este exponențial, a fost necesară mai întâi **liniarizarea** datelor prin aplicarea logaritmului.

Pentru calcul, am selectat intervalul de timp cuprins între indicii $i_5 = 6570$ și $i_6 = 8835$, corespunzător zonei în care dinamica sistemului este cel mai clar evidențiată. Ulterior, am aplicat metoda celor mai mici pătrate (regresia liniară) pentru a determina panta dreptei de aproximare, din care rezultă constanta de timp dominantă:

- Matricele `A_reg` și `B_reg` colectează informația din toate punctele selectate prin sumarea acestora.
- Vectorul `theta` reprezintă soluția sistemului, unde primul element (θ_1) este panta dreptei de regresie.
- Deoarece în urma liniarizării panta este egală cu inversul negativ al constantei de timp, am extras T_1 prin relația: $T_1 = -1/\theta_1$.

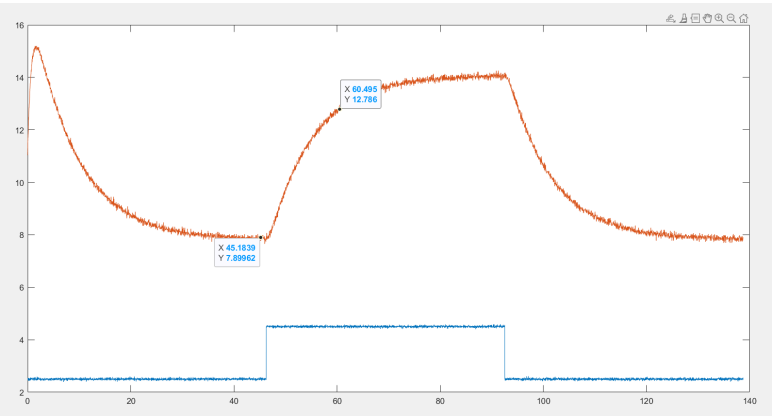

Figura 3: Identificarea intervalului $[i_5, i_6]$ pe panta de creștere a semnalului pentru calculul lui T_1.

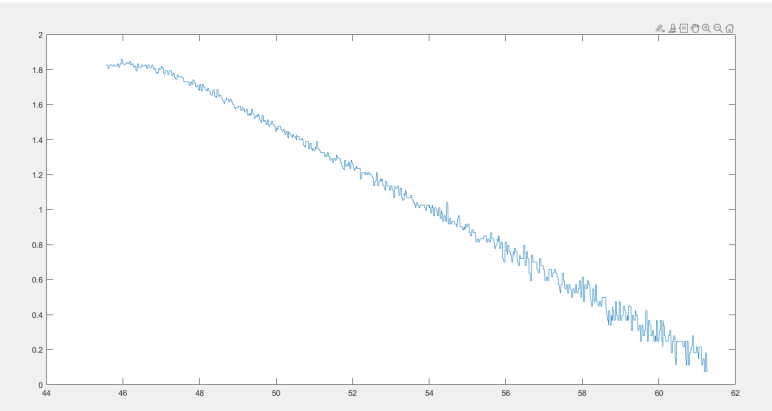

Figura 4: Reprezentarea grafică a datelor liniarizate pe intervalul $[i_5, i_6]$.

Codul MATLAB utilizat pentru această procedură este:

```
i5=6570;
i6=8835;
t_reg=t(i5:i6);
y_reg=log(yst-y(i5:i6));

figure
plot(t_reg,y_reg)

A_reg=[sum(t_reg.^2),sum(t_reg); sum(t_reg),length(t_reg)];
B_reg=[sum(y_reg.*t_reg);sum(y_reg)];
theta=inv(A_reg)*B_reg;
T1=-1/theta(1);
```

5 Determinarea constantei de timp nedominante (T_2)

Deoarece aceasta are o influență mai mică asupra formei generale a răspunsului, am utilizat o metodă de căutare iterativă (scanare) pentru a găsi valoarea care minimizează eroarea dintre datele experimentale și modelul teoretic.

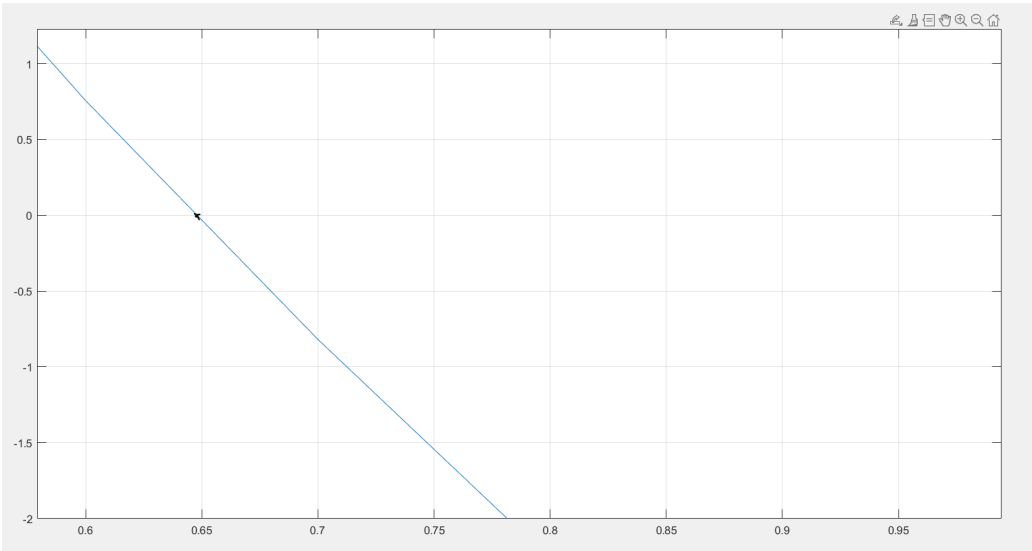

Figura 5: Identificarea constantei de timp nedominante.

Am ales T2=0.65 conform graficului in punctul 0.

```
i7=6764;
i8=7026;
Ti=t(i8)-t(i7);
T2vec=0.1:0.1:4.5;
Y_functie=T1*T2vec.*log(T2vec)-T2vec*(Ti+T1*log(T1))+T1*Ti;
figure
plot(T2vec,Y_functie)
grid
```

6 Modelarea și simularea răspunsului sistemului

În etapa finală, am definit modelul matematic al sistemului sub formă de funcție de transfer de ordinul II, utilizând valorile calculate anterior pentru K, T_1 și T_2.

Pentru a valida rezultatele, am simulat răspunsul acestui model la aceeași treaptă de intrare și am suprapus rezultatul peste datele experimentale. Suprapunerea aproape perfectă a celor două semnale confirmă faptul că identificarea parametrilor a fost realizată corect. Avem o eroare de doar 6.9%.

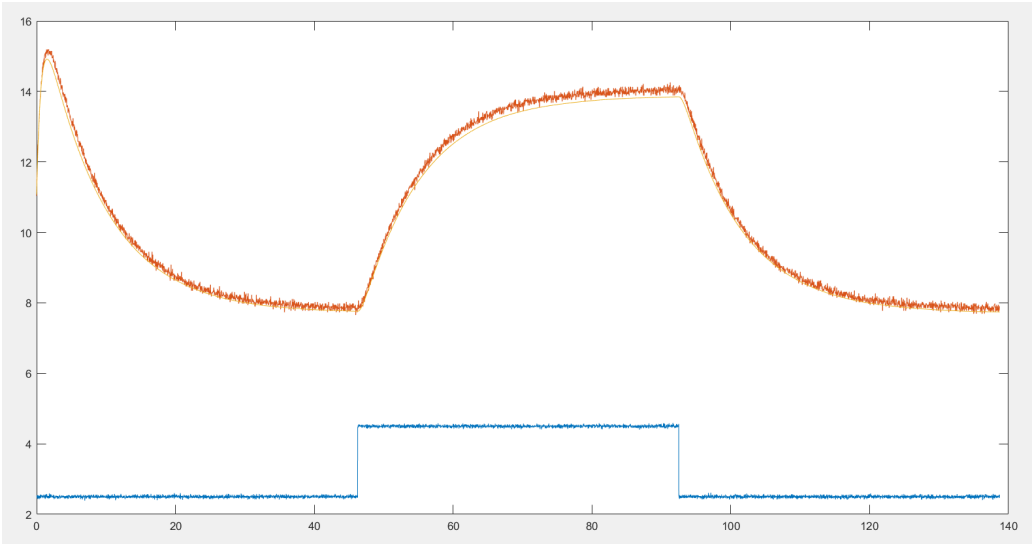

Figura 6: Compararea răspunsului real (y) cu cel simulat (ysim).

Codul MATLAB utilizat pentru simulare și reprezentare grafică este:

```
H=tf(K,conv([T1,1],[T2,1]));
ysim=lsim(H,u,t);
figure
plot(t,u,t,y,t,ysim)

A=[0,1;-1/T1/T2,-(1/T1+1/T2)];
B=[0;K/T1/T2];
C=[1,0];
D=0;

sys=ss(A,B,C,D);
ysim2=lsim(sys,u,t,[y(2),8]);
figure
plot(t,u,t,y,t,ysim2)
```

7 Anexa

```
%%
% Nume si prenume: Gulin Ghita Daniel
%
clearvars
clc
%% Magic numbers (replace with received numbers)
m = 6;
n = 10;
%% Process data (fixed, do not modify)
c1 = (1000+n*300)/10000;
c2 = (1.15+2*(m+n/10)/20);
a1 = 2*c2*c1;
a2 = c1;
b0 = (1.2+m+n)/5.5;
rng(m+10*n)
x0_slx = [2*(m/2+rand(1)*m/5); m*(n/20+rand(1)*n/100)];
%% Experiment setup (fixed, do not modify)
Ts = 10*c2/c1/1e4*1.5; % fundamental step size
Tfin = 30*c2/c1;
gain = 10;
umin = 0; umax = gain; % input saturation
ymin = 0; ymax = b0*gain/1.5; % output saturation
whtn_pow_in = 1e-6*5*(((m-1)*8+n/2)/5)/2*6/8; % input white noise power and sampling time
whtn_Ts_in = Ts*3;
whtn_seed_in = 23341+m+2*n;
q_in = (umax-umin)/pow2(10); % input quantizer (DAC)
whtn_pow_out = 1e-5*5*(((m-1)*25+n/2)/5)*6/80*(0.5+0.3*(m-2)); % output white noise power and sampling
whtn_Ts_out = Ts*5;
whtn_seed_out = 23342-m-2*n;
q_out = (ymax-ymin)/pow2(9); % output quantizer (ADC)
u_op_region = (m/2+n/5)/2; % operating point
%% Input setup (can be changed/replaced/deleted)
u0 = 0;        % fixed
ust = 2;     % must be modified (saturation)
t1 = 10*c2/c1; % recommended
%% Data acquisition (use t, u, y to perform system identification)
out = sim("GULIN_GhitaDaniel_circuit_hidraulic_regresie.slx");
t = out.tout;
u = out.u;
y = out.y;
plot(t,u,t,y)
shg
%% System identification
i1=6390;
i2=6515;
i3=12990;
i4=13175;
u0=mean(u(i1:i2));
ust=mean(u(i3:i4));
y0=mean(y(i1:i2));
yst=mean(y(i3:i4));
K= (yst-y0)/(ust-u0)
i5=6570;
i6=8835;
t_reg=t(i5:i6);
y_reg=log(yst-y(i5:i6));
```

```
figure
plot(t_reg,y_reg)
A_reg=[sum(t_reg.^2),sum(t_reg); sum(t_reg),length(t_reg)];
B_reg=[sum(y_reg.*t_reg);sum(y_reg)];
theta=inv(A_reg)*B_reg
T1=-1/theta(1)
i7=6764;
i8=7026;
%%
Ti=t(i8)-t(i7);
T2vec=0.1:0.1:4.5;
Y_functie=T1*T2vec.*log(T2vec)-T2vec*(Ti+T1*log(T1))+T1*Ti;
figure
plot(T2vec,Y_functie)
grid
%%
T2=0.65;
H=tf(K,conv([T1,1],[T2,1]));
ysim=lsim(H,u,t);
figure
plot(t,u,t,y,t,ysim)
A=[0,1;-1/T1/T2,-(1/T1+1/T2)];
B=[0;K/T1/T2];
C=[1,0];
D=0;
sys=ss(A,B,C,D);
ysim2=lsim(sys,u,t,[y(2),8]);
figure
plot(t,u,t,y,t,ysim2)
J=1/sqrt(length(t))*norm(y-ysim2)
eMPN=norm(y-ysim2)/norm(y-mean(y))*100
```

MINISTERUL EDUCAȚIEI NAȚIONALE
UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

Identificarea sistemelor de ordin II utilizând semnale de tip Chirp

Student: Ghiță Daniel Gulin
Profesor coordonator: Mircea Susca

1 Introducere

Proiectul propune identificarea modelelor matematice pentru sisteme de ordinul II utilizând estimarea răspunsului în frecvență. Spre deosebire de metodele bazate pe răspuns la treaptă, experimentele curente utilizează semnale de intrare de tip **Chirp** (semnal sinusoidal cu frecvență variabilă în timp). Validarea modelelor se realizează prin calculul erorii medii pătratice (J) și a erorii medii pătratice normalizate (ϵ_{MPN}), urmărind o precizie sub 10%.

2 Modelul Propus

Se propune modelarea sistemului utilizând o structură de ordinul II:

$$H(s) = \frac{K}{(T_1 s + 1)(T_2 s + 1)}, \quad K, T_1, T_2 > 0, \quad T_1 \gg T_2 \tag{2}$$

Problema de identificare: Obiectivul principal al acestui proiect constă în determinarea valorilor parametrilor K, T_1 și T_2 pe baza datelor experimentale obținute în urma aplicării semnalului de intrare de tip Chirp.

- **K** (Factorul de proporționalitate): determină amplificarea sistemului în regim staționar.
- T_1 (Constanta de timp dominantă): guvernează dinamica lentă a sistemului.
- T_2 (Constanta de timp nedominantă): reprezintă dinamica rapidă, fiind semnificativ mai mică decât T_1.

3 Calibrarea semnalului de intrare

Semnalul de tip Chirp este generat în jurul punctului de funcționare precizat de variabila `u_op_region`.

Conform principiilor de calibrare pentru semnale de tip VCO (Voltage Controlled Oscillator), s-au luat în considerare următoarele criterii pentru a asigura un experiment valid:

- **Gama de frecvențe** $[f_{min}, f_{max}]$: Aleasă astfel încât să permită reproducerea completă a răspunsului în frecvență al sistemului, incluzând zona de joasă frecvență și punctul caracteristic.
- **Durata experimentului** (T_{fin}): Stabilită pentru a asigura o variație suficient de lentă a frecvenței, astfel încât sistemul să poată atinge regimul cvasistaționar la fiecare frecvență intermediară.
- **Numărul de frecvențe intermediare**: Prin utilizarea unui semnal Chirp cu baleiere continuă, se asigură un număr ridicat de frecvențe intermediare ce ating spectrul necesar pentru excitarea sistemului și identificarea precisă a momentului de defazaj de $-\pi/2$.

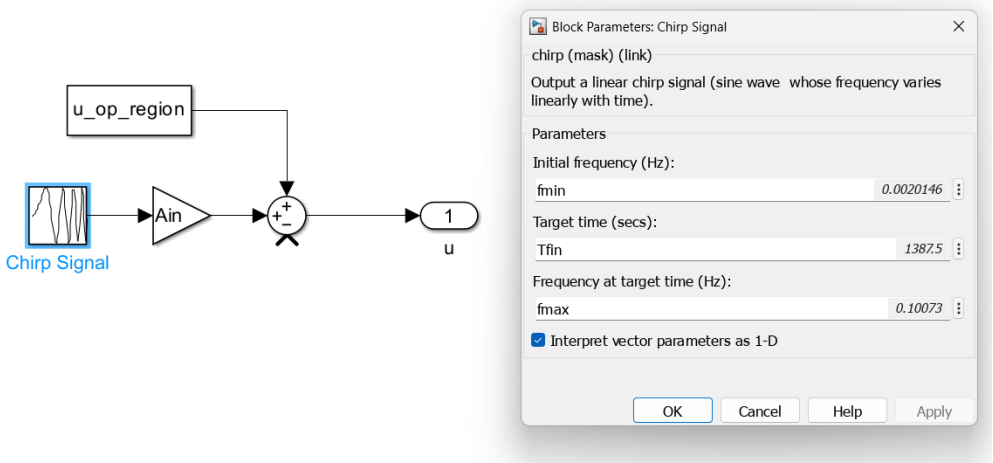

Figura 1: Semnalul de intrare de tip Chirp utilizat.

4 Identificarea parametrilor

4.1 Determinarea Factorului de Proporționalitate (K)

$$K \approx \frac{\bar{y}_{st}}{\bar{u}_{st}} \tag{3}$$

Cod MATLAB:

```
yst=(9.44+6.23)/2;
ust=2.5;
K=yst/ust;
```

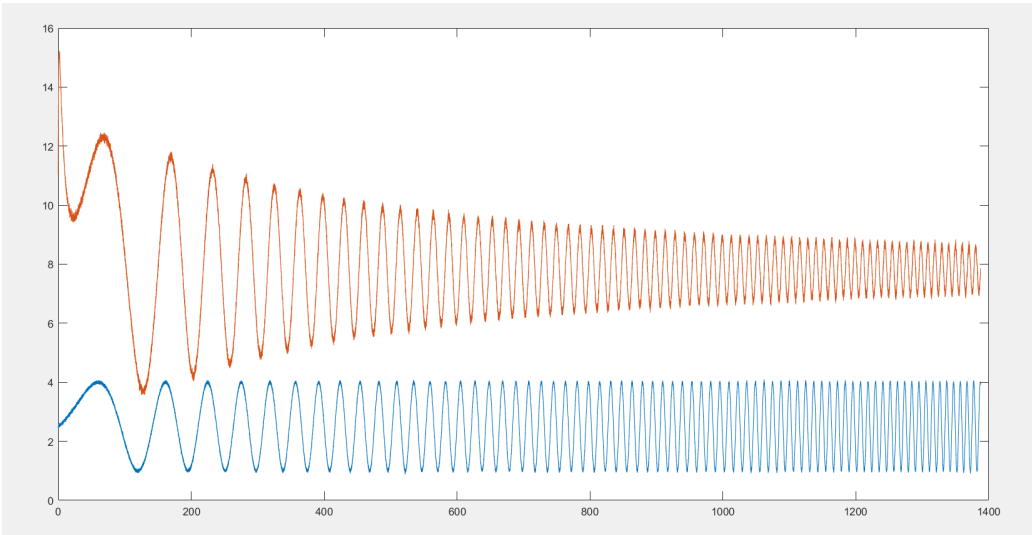

Figura 2: Semnalul de intrare de tip Chirp și răspunsul sistemului indicând variația frecvenței în timp.

4.2 Determinarea pulsației de cuadratură (w_1)

Pulsația de cuadratură reprezintă frecvența unghiulară la care sistemul introduce un defazaj de exact $-90°$. Aceasta a fost determinată prin măsurarea semiperioadei (Δt) a semnalului de intrare în zona unde trecerile prin zero ale acestuia coincid cu valorile extreme ale ieșirii.

- **Momente de timp:** $t_1 = 882.172$ și $t_2 = 874.548$.
- **Formula de calcul:**

$$w_1 = \frac{\pi}{t_2 - t_1} \tag{4}$$

- **Rezultat:** Pentru o diferență de timp de aproximativ 3.7 s (conform codului), s-a obținut $w_1 \approx 0.41$ rad/s.

4.3 Calculul diferenței de timp (ΔT_1)

Diferența de timp reprezintă decalajul temporal (întârzierea) dintre semnalul de intrare și cel de ieșire.

- **Puncte extrase:** $t_{u_peak} = 875.027$ (minim intrare) și $t_{y_peak} = 871.322$ (minim ieșire).
- **Calcul:**

$$\Delta T_1 = t_{y_peak} - t_{u_peak} = 875.027 - 871.322 = 3.705 \tag{5}$$

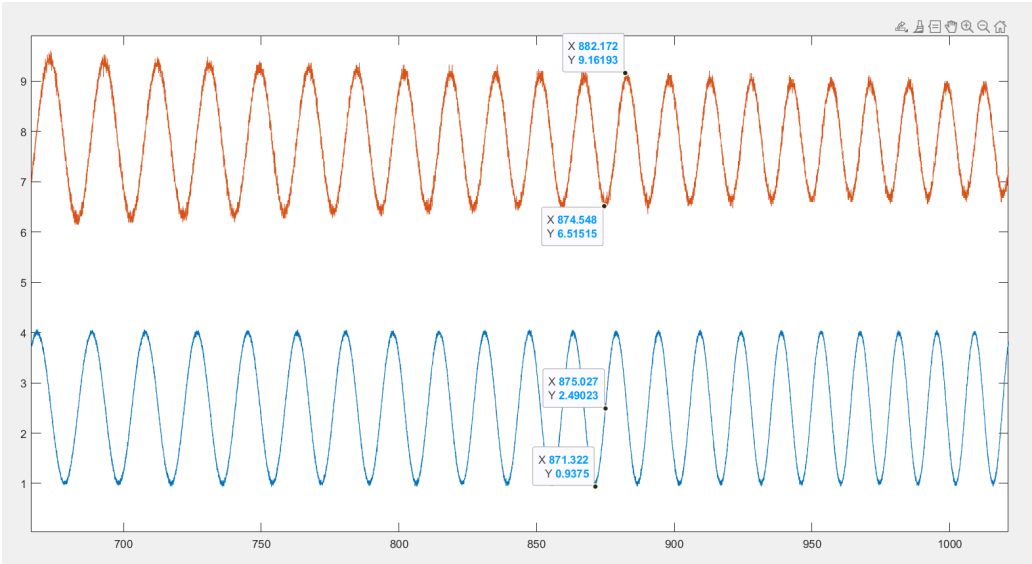

Figura 3: Identificarea vizuală a punctelor de cuadratură și a decalajului temporal ΔT_1.

4.4 Determinarea defazajului (ϕ_1)

Defazajul confirmă poziționarea sistemului în zona de cuadratură. Acesta a fost calculat prin raportarea decalajului temporal la perioada locală a oscilației.

- **Formula:**

$$\phi_1 = -w_1 \cdot \Delta T_1 \tag{6}$$

  Valoarea rezultată în radiani a fost convertită în grade utilizând funcția `rad2deg`, obținându-se o valoare de aproximativ $-90°$, ceea ce validează alegerea punctelor de identificare.

4.5 Extragerea constantelor de timp (T_1 și T_2)

Codul transformă datele geometrice extrase din graficul Chirp (amplitudini și pulsație) în parametrii matematici (K, ζ, ω_n) necesari pentru construcția funcției de transfer de ordinul II a sistemului. Procesul de calcul este implementat în următoarea secvență MATLAB:

```
Ay = (9.16 - 6.51) / 2;
Au = 1.5;
M = Ay / Au;
Im = -M;
wn = w1;
zeta = -K / 2 / Im;
H = tf([K * wn^2], [1, 2 * zeta * wn, wn^2]);
zpk(H)
T1 = 1 / 0.1272;
T2 = 1 / 1.335;
```

- **Metodă:** S-a utilizat comanda `zpk(H)` pentru a identifica polii reali ai sistemului (p_1, p_2) din forma polinomială a numitorului. Această factorizare permite trecerea de la forma canonică a sistemului de ordinul II la forma cu constante de timp.
- **Rezultate:** Din valorile polilor rezultați la numitorul funcției de transfer, s-au extras constantele de timp ca inverse ale acestora:
  - Polul $p_1 = 0.1272 \Rightarrow T_1 = \frac{1}{0.1272} \approx 7.86$;
  - Polul $p_2 = 1.335 \Rightarrow T_2 = \frac{1}{1.335} \approx 0.75$.

5 Validarea modelului

În etapa finală, am definit modelul matematic al sistemului sub formă de reprezentare în spațiul stărilor, utilizând valorile identificate pentru câștigul K și constantele de timp T_1 și T_2.

Pentru a valida rezultatele, am simulat răspunsul acestui model la semnalul de intrare de tip Chirp utilizând funcția `lsim` și am suprapus rezultatul peste datele experimentale. Calculul indicelui de performanță a generat o valoare $EMPN = 7.2940\%$.

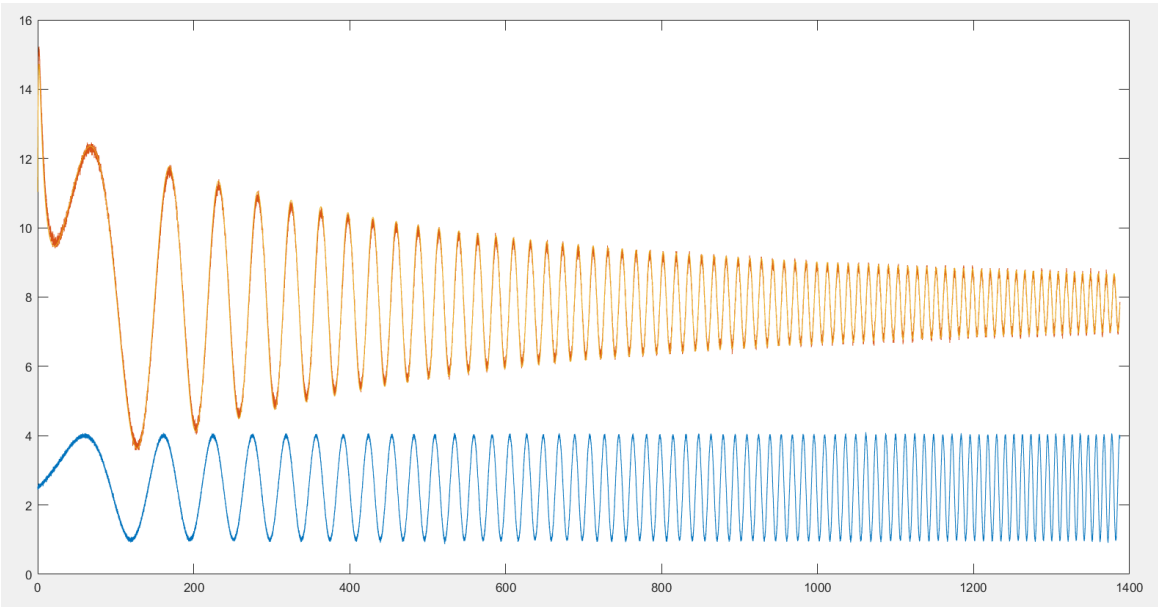

Figura 4: Rezultatele validării modelului: suprapunerea semnalului experimental y cu semnalul simulat $ysim2$ ($EMPN = 7.2940\%$) si J=3.0339e-04.

Codul MATLAB utilizat pentru validarea sistemului:

```
A=[0,1;-1/T1/T2,-(1/T1+1/T2)];
B=[0;K/T1/T2];
C=[1,0];
D=0;

sys=ss(A,B,C,D);
ysim2=lsim(sys,u,t,[y(1),7]);

J=1/sqrt(length(t)*norm(y-ysim2))
empn = norm(y-ysim2)/norm(y-mean(y))*100
figure
plot(t,u,t,y,t,ysim2)
```

Anexa:

```
%%
% Nume si prenume: Gulin Ghita Daniel
%

clearvars
clc

%% Magic numbers (replace with received numbers)
m = 6;
n = 10;

%% Process data (fixed, do not modify)
c1 = (1000+n*300)/10000;
c2 = (1.15+2*(m+n/10)/20);
a1 = 2*c2*c1;
a2 = c1;
b0 = (1.2+m+n)/5.5;

rng(m+10*n)
x0_slx = [2*(m/2+rand(1)*m/5); m*(n/20+rand(1)*n/100)];

%% Experiment setup (fixed, do not modify)
Ts = 10*c2/c1/1e4*1.5; % fundamental step size
Tfin = 30*c2/c1*10; % simulation duration

gain = 10;
umin = 0; umax = gain; % input saturation
ymin = 0; ymax = b0*gain/1.5; % output saturation

whtn_pow_in = 1e-6*5*(((m-1)*8+n/2)/5)/2*6/8; % input white noise power and sampling time
whtn_Ts_in = Ts*3;
whtn_seed_in = 23341+m+2*n;
q_in = (umax-umin)/pow2(10); % input quantizer (DAC)

whtn_pow_out = 1e-5*5*(((m-1)*25+n/2)/5)*6/80*(0.5+0.3*(m-2)); % output white noise power and sampling
whtn_Ts_out = Ts*5;
whtn_seed_out = 23342-m-2*n;
q_out = (ymax-ymin)/pow2(9); % output quantizer (ADC)

u_op_region = (m/2+n/5)/2; % operating point
```

```
%% Input setup (can be changed/replaced/deleted)
u0 = 0;     % fixed
ust = 2.5;  % must be modified (saturation)
t1 = 12/a1; % recommended
wf=1/7.9; %1/T1, T1 dominant; T1 usor de aproximat din rasp la treapta
fmin=wf/2/pi/10;
fmax=wf/2/pi*5;
Ain=1.5; %arbitrar, suficient de mare incat sa depaseasca nivelul zgomotului

%% Data acquisition (use t, u, y to perform system identification)
out = sim("GULIN_GhitaDaniel_circuit_hidraulic_chirp.slx");

t = out.tout;
u = out.u;
y = out.y;

plot(t,u,t,y)
shg

%% System identification
yst=(9.44+6.23)/2;
ust=2.5;
K=yst/ust;
w1=pi/(882.172-874.548)
DeltaT1=(875.027-871.322)
phi1 = (rad2deg(-w1*DeltaT1))

Ay=(9.16-6.51)/2
Au=1.5;
M=Ay/Au
Im=-M
wn=w1
zeta=-K/2/Im
H=tf([K*wn^2],[1,2*zeta*wn,wn^2])
zpk(H)
T1=1/0.1272
T2=1/1.335

A=[0,1;-1/T1/T2,-(1/T1+1/T2)];
B=[0;K/T1/T2];
C=[1,0];
D=0;

sys=ss(A,B,C,D);
ysim2=lsim(sys,u,t,[y(1),7]);

J=1/sqrt(length(t)*norm(y-ysim2))
empn = norm(y-ysim2)/norm(y-mean(y))*100
figure
plot(t,u,t,y,t,ysim2)
```

MINISTERUL EDUCAȚIEI NAȚIONALE

UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

Calibrarea semnalelor pseudoaleatoare binare (SPAB)

Student: Ghiță Daniel Gulin
Profesor coordonator: Mircea Susca

1 Introducere

Lucrarea curenta de proiect propune proiectarea si calibrarea semnalelor pseudoaleatoare binare (SPAB) pentru identificarea sistemelor dinamice utilizand metode parametrice. Se propune un proces neliniar de studiu si se analizeaza problema matematica de calibrare a unui semnal de intrare adecvat pentru evidentierea unei dinamici aproximativ liniare.

2 Calibrarea intrării - Convertor DC-DC ridicător de tensiune

Proiectarea semnalului de intrare a presupus combinarea a două tipuri de semnale:

- **Semnalul trapezoidal:** utilizat pentru a stabili punctele de funcționare ale sistemului neliniar.
- **Semnalul SPAB:** utilizat pentru a genera o gamă largă de frecvențe.

Parametrii de intrare:

- `Ts`: perioada de eșantionare fundamentală, calculată în funcție de punctul de funcționare($25\mu s$).
- `t1`: timpul de așteptare pentru aplanarea condițiilor inițiale (0.3 s).
- `Tfin`: durata totală a experimentului Simulink.
- **u_star** ($u^* = 0.6$)**:** Reprezintă factorul de umplere mediu (punctul de funcționare) în jurul căruia sistemul este liniarizat.
- **delta** ($\delta = 0.02$)**:** Reprezintă mărimea treptei trapezoidale care separă cele două paliere (identificare și validare).
- **delta_spab** ($\delta_{SPAB} = 0.015$)**:** Reprezintă amplitudinea semnalului pseudoaleator binar.
- **N = 5**: numărul de biți ai registrului de deplasare.
- **p = 480** (calculat): divizorul de frecvență care scalează viteza SPAB în funcție de timpul de urcare estimat al sistemului.
- **DeltaT**: durata totală a semnalului SPAB.
- **input_LUT_dSPACE** reprezintă semnalul de intrare generat pentru procesul de identificare

Modificari realizate in Simulink:

1. A fost adaugat un bloc de tip From_Workspace. Acest bloc furnizează semnalul de intrare `input_LUT_dSpace`, generat anterior prin funcția `generate_input_signal`.

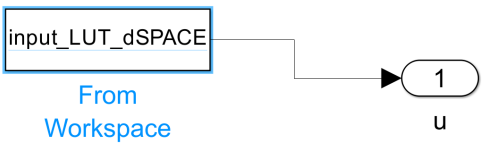

Figura 1: Integrarea blocului `From Workspace` în schema Simulink.

2. A fost setata perioada de esantionare Ts in blocul From Workspace si activarea Interpolate data

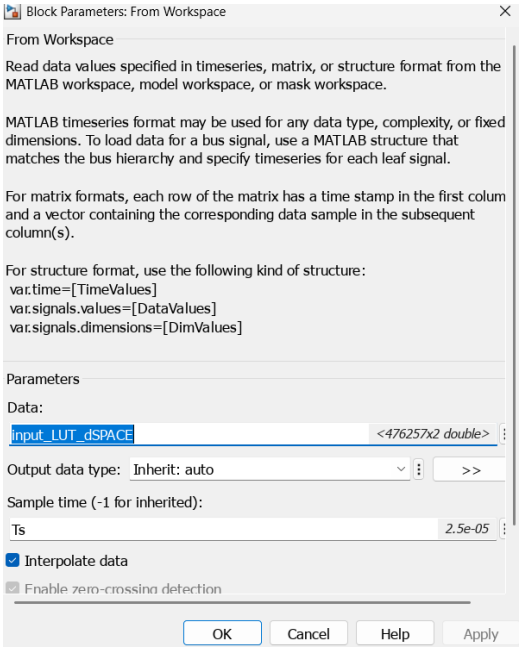

Figura 2: Configurarea perioadei de eșantionare T_s

3. Stop Time a fost setat la Tfin

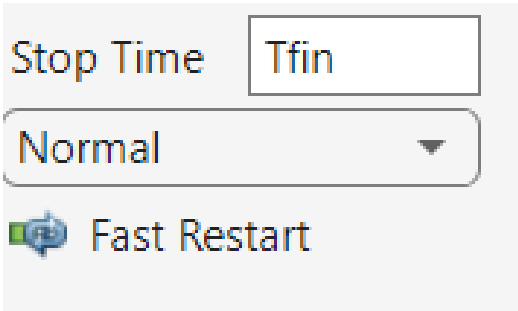

Figura 3: Setarea timpului final de simulare T_{fin}.

out = sim("convertor_R2022b.slx"); - Lansează în execuție simularea modelului Simulink și salvează toate rezultatele în obiectul `out`.
Extragerea datelor:

```
t = out.tout;
u = out.u;
y = out.y;
```

Vizualizarea semnalelor:

```
subplot(211)
plot(t,u)
subplot(212)
plot(t,y)
```

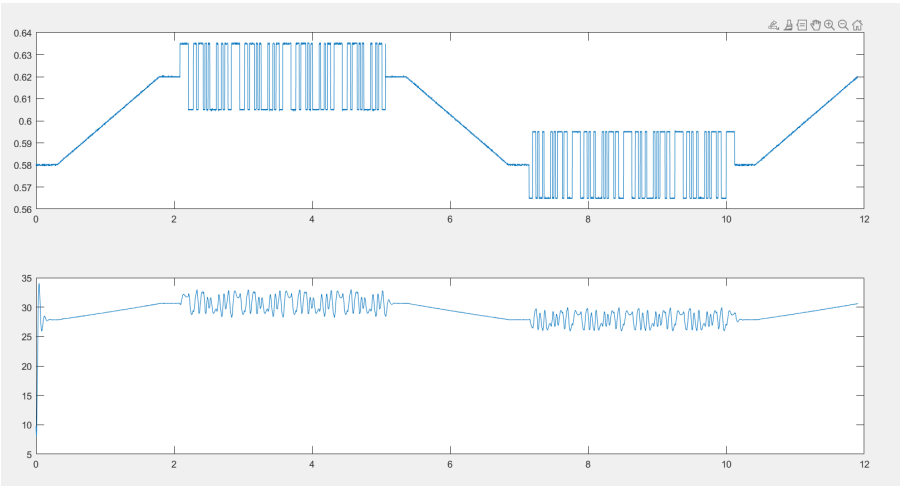

Figura 4: Reprezentarea grafică a semnalelor: intrarea $u(t)$ și ieșirea $y(t)$.

Codul MATLAB utilizat:

```
%

clearvars
clc

%% Magic numbers (replace with received numbers)
m = 6;
n = 10;

%% Process data and experiment setup (fixed, do not modify)
u_star = 0.15+n*0.045; % trapeze + PRBS amplitudes
delta = 0.02;
delta_spab = 0.015;

E = 12;  % converter source voltage

umin = 0; umax = 0.98; % input saturation
assert(u_star < umax-0.1)
ymin = 0; ymax = 1/(1-u_star)*E*2; % output saturation

% circuit components + parasitic terms
R = 15;
rL = 10e-3;
rC = 0.2;
rDS1 = 0.01;
rDS2 = 0.01;
Cv = 600e-6/3*m;
Lv = 40e-3*3/m;

% (iL0,uC0)
rng(m+10*n)
```

```
Ts = 1e-5*(1+2*(u_star-0.15)/u_star); % fundamental step size
Ts = round(Ts*1e6)/1e6;

% input white noise power and sampling time
whtn_pow_in = 1e-11*(Ts*1e4)/2;
whtn_Ts_in = Ts*2;
whtn_seed_in = 23341+m+2*n;
q_in = (umax-umin)/pow2(11); % input quantizer (DAC)

% output white noise power and sampling time
whtn_pow_out = 1e-7*E*(Ts*1e4/50)*(1+(50*u_star)*(u_star-0.15))/3;
whtn_Ts_out = Ts*2;
whtn_seed_out = 23342-m-2*n;
q_out = (ymax-ymin)/pow2(11); % output quantizer (ADC)

meas_rep = 13+ceil(n/2); % data acquisition hardware sampling limitation

%% Input setup (can be changed/replaced/deleted)
t1=0.3;
tr=0.04*3; %timp de urcare, aproximativ timp de urcare
N=5; %nr de biti a SPABului (la alegere luat)
p=round(tr/N/Ts);
DeltaT=p*(2^N-1)*Ts*4;

[input_LUT_dSPACE,Tfin] = generate_input_signal(Ts,t1,DeltaT,N,p,u_star,delta,delta_spab);



%% Data acquisition (use t, u, y to perform system identification)
out = sim("convertor_R2022b.slx");

t = out.tout;
u = out.u;
y = out.y;

subplot(211)
plot(t,u)
subplot(212)
plot(t,y)
```

MINISTERUL EDUCAȚIEI NAȚIONALE
UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

Identificarea sistemelor utilizând metode parametrice

Student: Ghiță Daniel Gulin
Profesor coordonator: Mircea Susca

1 Introducere

Scopul principal al acestei lucrări este aplicarea metodologiilor de identificare parametrică pentru construirea, rafinarea și validarea modelelor matematice ale unui proces. Proiectul utilizează datele experimentale utilizate anterior (pentru convertorul DC-DC boost), trecând de la metodele grafice la utilizarea funcțiilor specializate din *System Identification Toolbox*.

1.1 Construirea seturilor de date de identificare și validare

Pentru a izola regimul dinamic util, am ales manual indicii de început și de sfârșit pentru cele două seturi de date, așa cum se observă în Figura 1:

- **Selectia secventelor:** Am aplicat o decimare cu factorul $N_r = 20$ pentru a reduce densitatea mare de eșantioane. Decimarea secventelor de identificare si validare:

```
t_id=t(i1:Nr:i2);
u_id=u(i1:Nr:i2);
y_id=y(i1:Nr:i2);
t_vd=t(i3:Nr:i4);
u_vd=u(i3:Nr:i4);
y_vd=y(i3:Nr:i4);
```

- **Eliminarea componentei continue (Scăderea mediei):** S-a aplicat centrarea semnalelor de intrare și ieșire prin scăderea valorii medii (`u - mean(u)`), operație necesară pentru a lucra în coordonate incrementale în jurul punctului de funcționare.

```
u_id=u_id-mean(u_id);
y_id=y_id-mean(y_id);
u_vd=u_vd-mean(u_vd);
y_vd=y_vd-mean(y_vd);
```

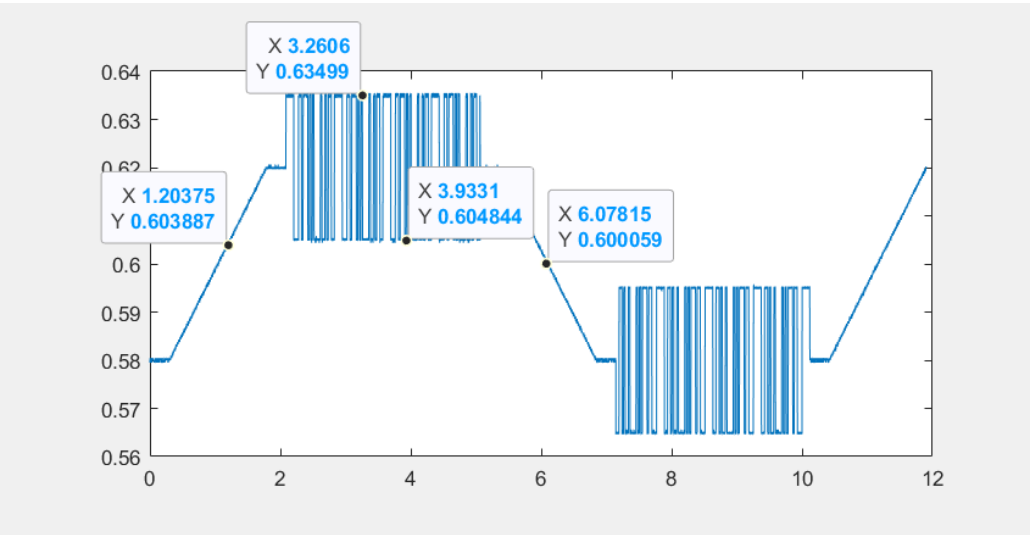

Figura 1: Indicii alesi pentru procesul de identificare și validare.

1.2 Vizualizarea datelor de identificare și validare

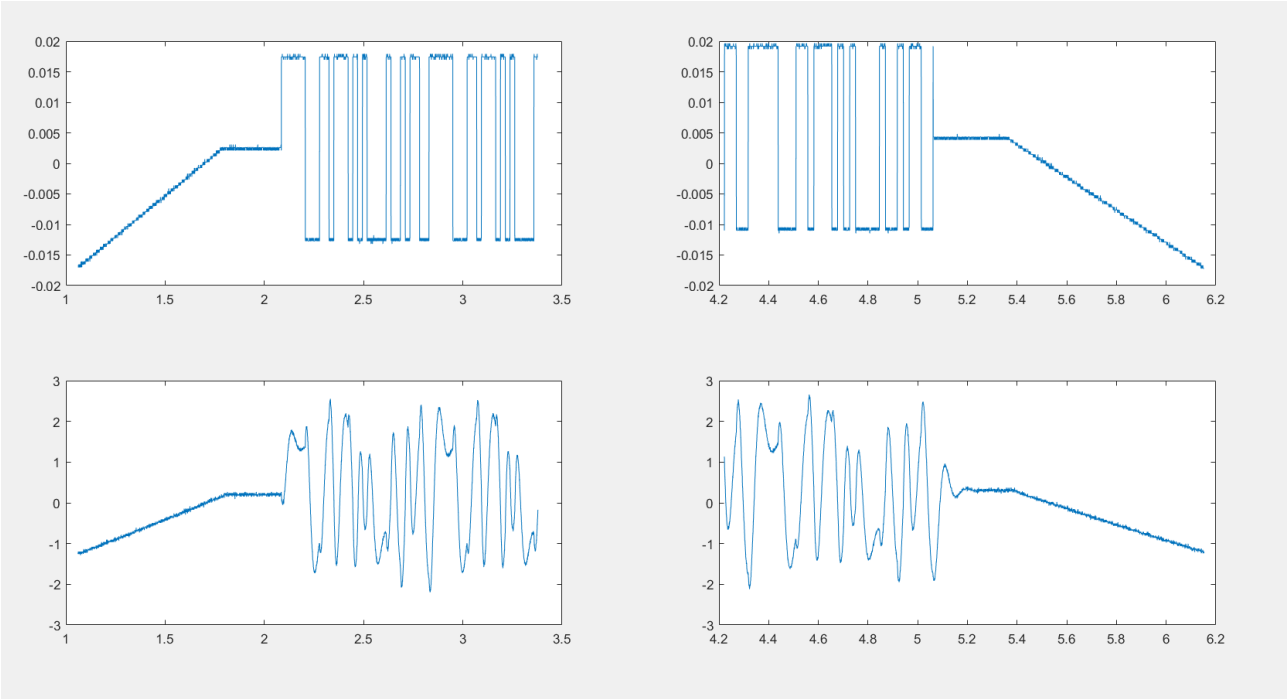

Figura 2: Reprezentarea semnalelor de intrare și ieșire pentru subseturile de date alese.

Explicația fiecărui grafic din figură este următoarea:

- **Subplot(221) - Intrarea de identificare (u_{id}):** Reprezintă semnalul de intrare (excitația) în fereastra $[i_1, i_2]$.
- **Subplot(223) - Ieșirea de identificare (y_{id}):** Reprezintă răspunsul sistemului la intrarea de identificare.
- **Subplot(222) - Intrarea de validare (u_{vd}):** Reprezintă semnalul de intrare pentru a doua fereastră, $[i_3, i_4]$.
- **Subplot(224) - Ieșirea de validare (y_{vd}):** Reprezintă răspunsul real al sistemului în fereastra de validare.
- **Datele de identificare (`dat_id`):** Reprezintă eșantioanele dintre i_1 și i_2, fiind folosite pentru calculul parametrilor modelelor.
- **Datele de validare (`dat_vd`):** Reprezintă eșantioanele dintre i_3 și i_4, fiind folosite doar pentru a testa precizia modelelor pe date noi.

```
dat_id = iddata(y_id, u_id, t_id(2) - t_id(1));
dat_vd = iddata(y_vd, u_vd, t_vd(2) - t_vd(1));
```

1.3 1. Modelul ARX [6, 6, 1]

Acest model a fost estimat utilizând funcția `arx`.

- **Structura:** S-a ales o configurație cu $na = 6, nb = 6$ și o întârziere $nk = 1$, s-a ales un ordin mai mare pentru a reprezenta mai bine detaliile.
- **Justificare:** Ordinul ridicat (6) a fost selectat pentru a oferi modelului flexibilitatea necesară în captarea dinamicii sistemului în prezența zgomotului, asigurând o estimare robustă a polilor dominanți.
- **Validare:** Modelul este validat prin testul de autocorelație a reziduurilor si prezinta o suprapunere de 65%.

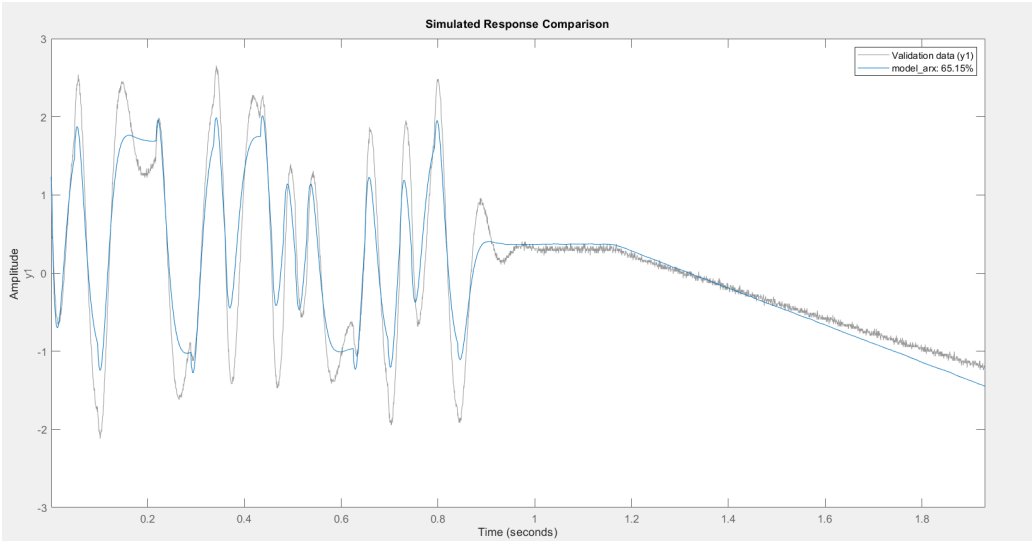

Figura 3: Validarea modelului ARX prin compararea ieșirii simulate cu ieșirea măsurată.

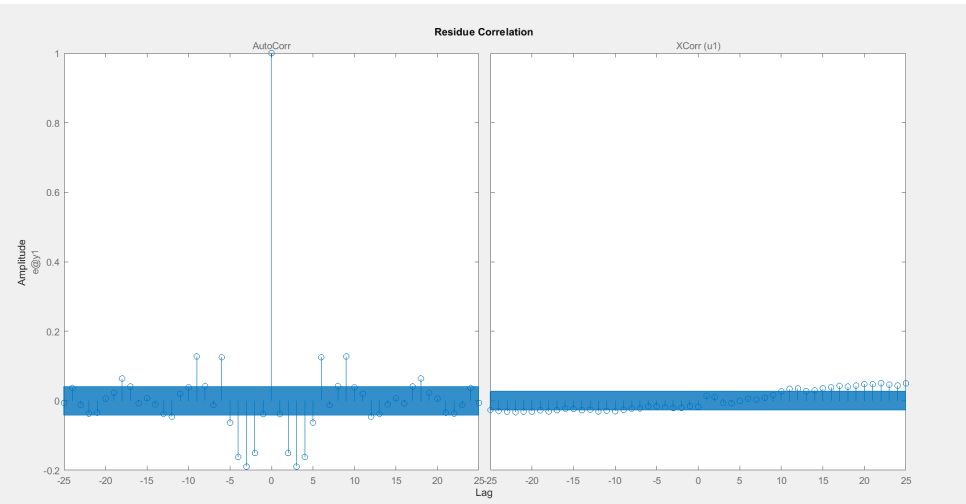

Figura 4: Analiza statistică a reziduurilor pentru modelul ARX (Funcția `resid`).

1.4 2. Modelul ARMAX [4, 4, 2, 1]

Structura ARMAX a fost implementată pentru a include o modelare explicită a perturbărilor ce afectează procesul si este un model valid prin autocorelatie.

- **Structura:** Parametrii utilizați sunt $[na, nb, nc, nk] = [4, 4, 2, 1]$.
- **Justificatie:** Am ales un ordin mai mic decat la ARX deoarece ARMAX este mai precis si are in plus un parametru (aici 2) care se ocupa de curatarea zgomotului.
- **Validare:** Avem o suprapunere de 94%.

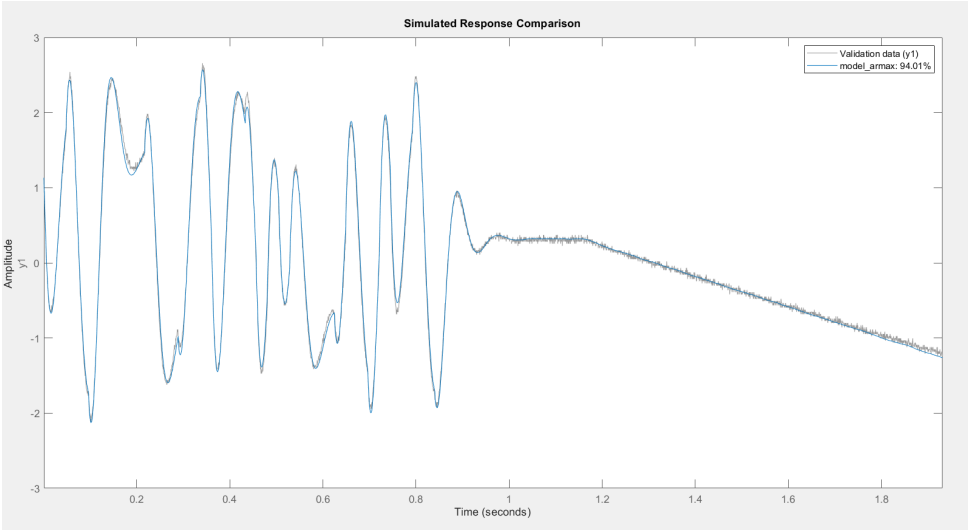

Figura 5: Validarea modelului ARMAX: Compararea ieșirii estimate cu datele de validare.

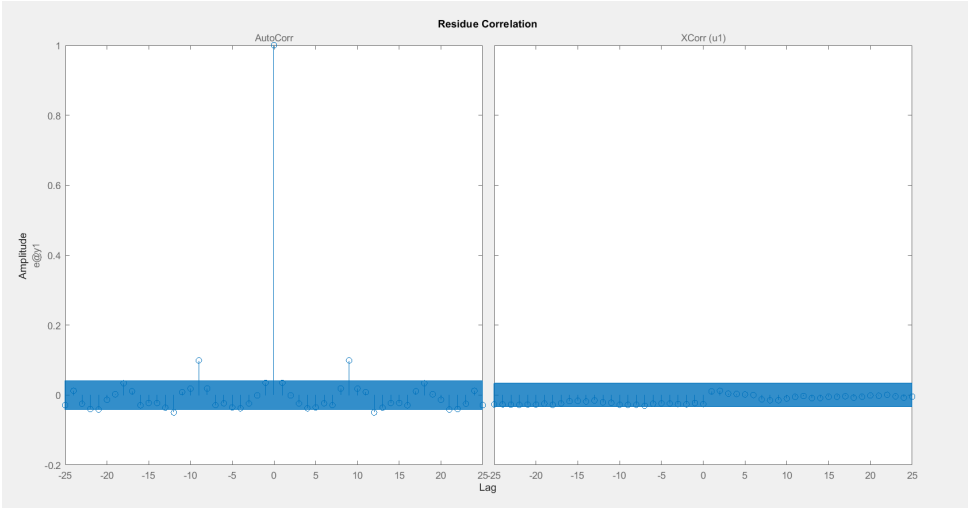

Figura 6: Analiza statistică a reziduurilor pentru modelul ARMAX (`resid`).

1.5 3. Modelul Output-Error (OE) [2, 2, 1]

Modelul OE este un model valid prin intercorelatie.

- **Structura:** S-a optat pentru $nb = 2, nf = 2$ și $nk = 1$.
- **Justificare:** Această structură minimizează direct eroarea dintre ieșirea reală și cea simulată a modelului, am ales ordin mai mic deoarece reprezinta corect graficul.
- **Validare:** Avem o suprapunere de 93.7%.

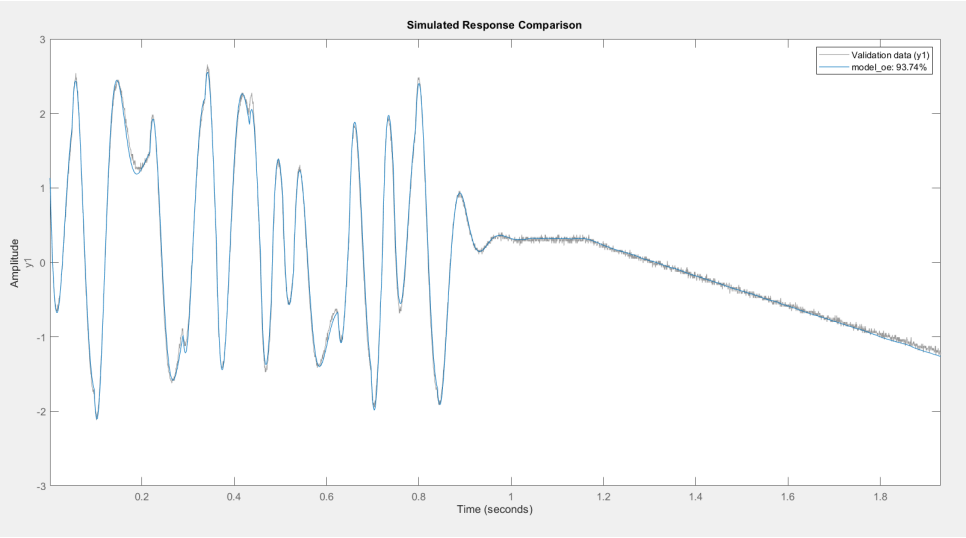

Figura 7: Validarea modelului OE: Compararea ieșirii reale cu ieșirea modelului pe datele de validare.

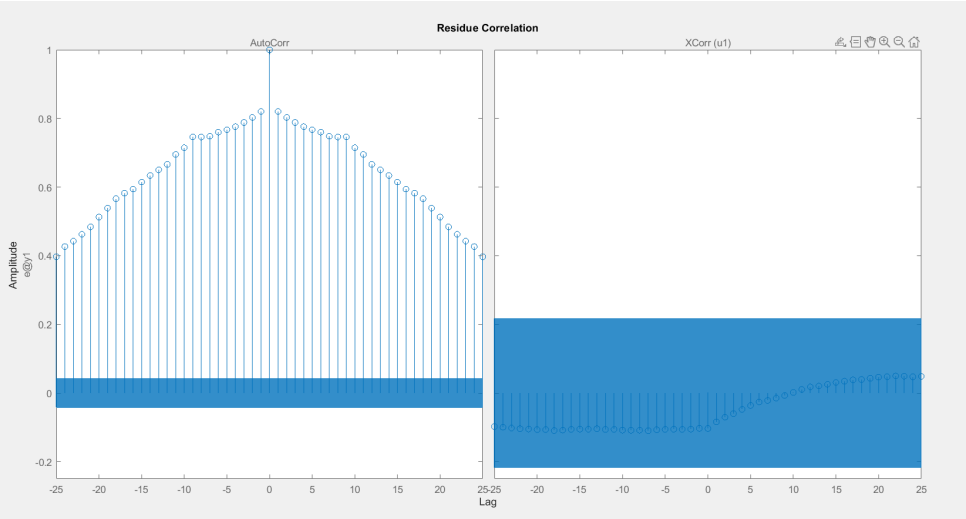

Figura 8: Analiza statistică a reziduurilor pentru modelul OE (`resid`).

1.6 4. Modelul în spațiul stărilor (N4SID)

Acesta este un model valid de alta natura fata de cele de mai sus.

- **Configurație:** S-a impus un ordin al sistemului $nx = 8$.
- **Justificare:** Am ales acest ordin pentru ca o valoare mai mica rezulta o eroare mai mare decat dorita.
- **Validare:** Avem o suprapunere de 93.9%.

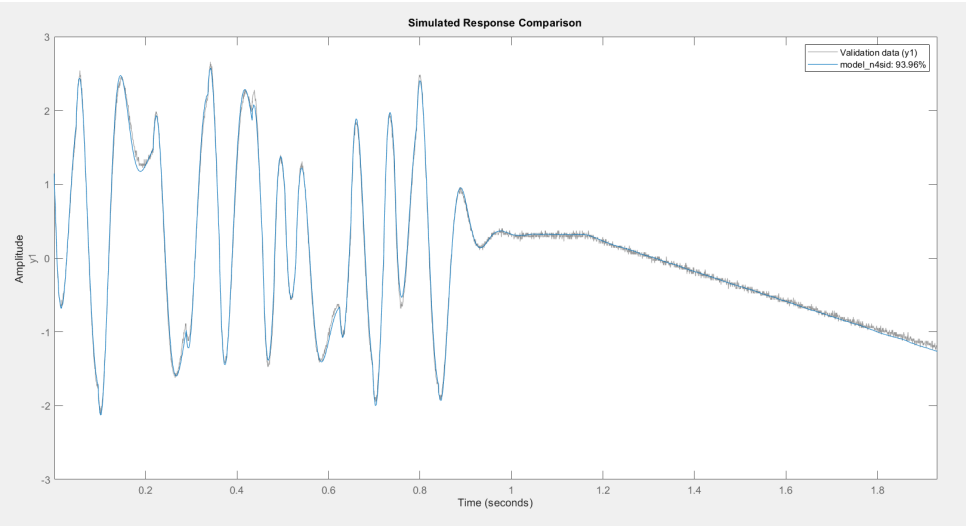

Figura 9: Validarea modelului N4SID: Gradul de suprapunere între ieșirea sistemului și cea a modelului.

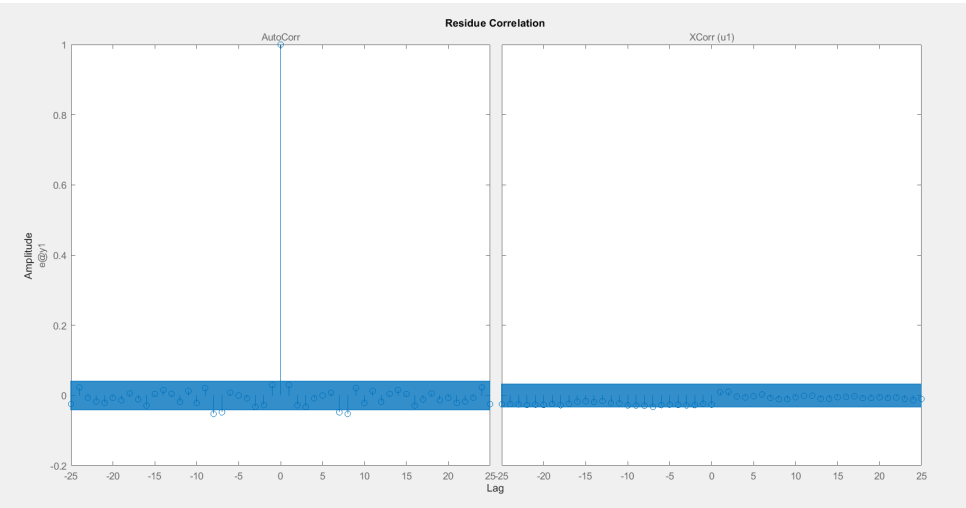

Figura 10: Analiza statistică a reziduurilor pentru modelul N4SID (`resid`).

1.7 5. Modelul în spațiul stărilor (SSEST)

Ultima metodă aplicată este estimarea iterativă în spațiul starilor care utilizează un algoritm de optimizare pentru a rafina parametrii modelului pornind de la o estimare inițială."

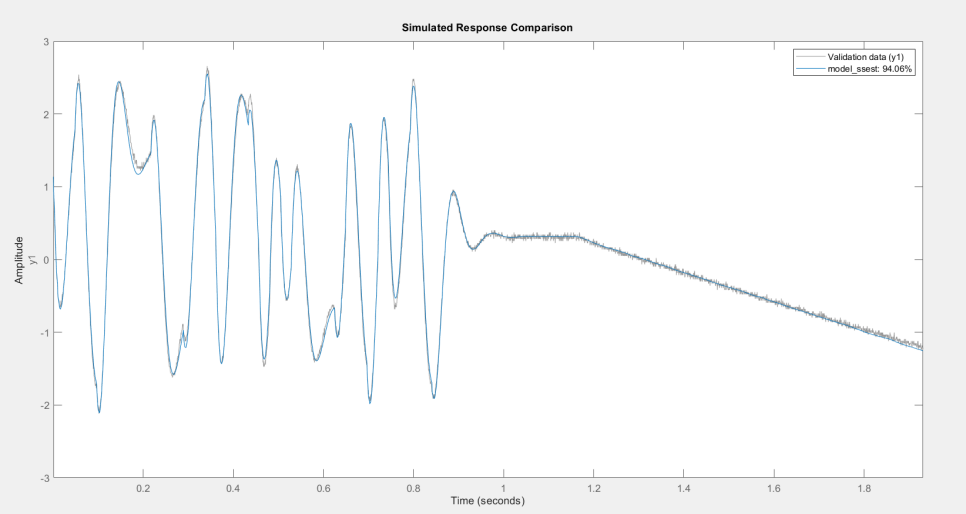

Figura 11: Validarea modelului SSEST: Compararea ieșirii măsurate cu cea simulată.

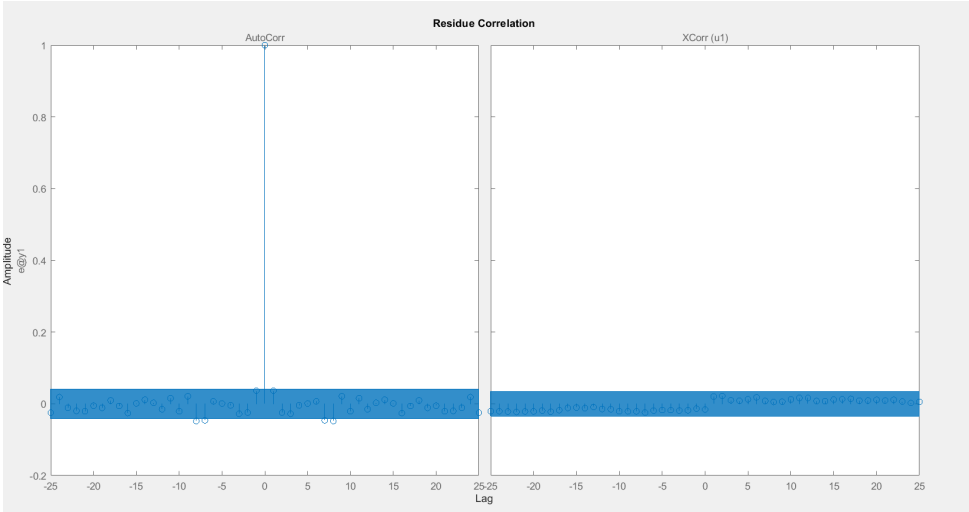

Figura 12: Analiza statistică a reziduurilor pentru modelul SSEST (`resid`).

- **Configurație:** S-a impus un ordin al sistemului $nx = 4$.
- **Justificare:** Am ales acest ordin pentru ca o valoare mai mica rezulta ca graficul de la resid nu era potrivit.
- **Validare:** Avem o suprapunere de 94%.

Stabilitate: Toți polii modelului au partea reală negativă. Asta înseamnă că sistemul este stabil:

```
From input "u1" to output "y1":
  -7203.5 (s-107.8) (s^2 + 1516s + 1.901e06)
-----------------------------------------------
(s^2 + 59.47s + 6141) (s^2 + 1397s + 3.172e06)
```

Analiza regimului staționar a fost realizată prin calcularea factorului de proporționalitate K, utilizând teorema valorii finale ($s \to 0$). Pentru modelul de ordinul 4 validat, s-a obținut valoarea $K = [75]$. Această valoare indică sensibilitatea tensiunii de ieșire la variațiile factorului de umplere în punctul de funcționare ales.

2 Concluzii

În urma analizelor comparative, s-a stabilit că modelul de tip **ARMAX [4 4 2 1]** reprezintă cea mai bună aproximare a convertorului DC-DC analizat.

Argumentele care susțin această alegere sunt:

- **Performanță:** A obținut cel mai ridicat indicator FIT (94%), oferind o suprapunere aproape ideală peste datele de validare.
- **Testul reziduurilor:** Se observă că majoritatea valorilor se află în interiorul intervalului. Acest lucru demonstrează că modelul a extras corect dinamica sistemului.

Codul MATLAB utilizat:

```
  %%
% Nume si prenume: Gulin Ghita-Daniel
%

clearvars
clc

%% Magic numbers (replace with received numbers)
m = 6;
n = 10;

%% Process data and experiment setup (fixed, do not modify)
u_star = 0.15+n*0.045; % trapeze + PRBS amplitudes
delta = 0.02;
delta_spab = 0.015;

E = 12;  % converter source voltage

umin = 0; umax = 0.98; % input saturation
assert(u_star < umax-0.1)
ymin = 0; ymax = 1/(1-u_star)*E*2; % output saturation

% circuit components + parasitic terms
R = 15;
rL = 10e-3;
rC = 0.2;
rDS1 = 0.01;
rDS2 = 0.01;
Cv = 600e-6/3*m;
Lv = 40e-3*3/m;

% (iL0,vC0)
```

```
rng(m+10*n)
x0_slx = [(-1)^(n+1)*E/R,E/3/(1-u_star)];

Ts = 1e-5*(1+2*(u_star-0.15)/u_star); % fundamental step size
Ts = round(Ts*1e6)/1e6;

% input white noise power and sampling time
whtn_pow_in = 1e-11*(Ts*1e4)/2;
whtn_Ts_in = Ts*2;
whtn_seed_in = 23341+m+2*n;
q_in = (umax-umin)/pow2(11); % input quantizer (DAC)

% output white noise power and sampling time
whtn_pow_out = 1e-7*E*(Ts*1e4/50)*(1+(50*u_star)*(u_star-0.15))/3;
whtn_Ts_out = Ts*2;
whtn_seed_out = 23342-m-2*n;
q_out = (ymax-ymin)/pow2(11); % output quantizer (ADC)

meas_rep = 13+ceil(n/2); % data acquisition hardware sampling limitation

%% Input setup (can be changed/replaced/deleted)
t1=0.3;
tr=0.04*3; %timp de urcare, aproximativ timp de urcare
N=5; %nr de biti a SPABului (la alegere luat)
p=round(tr/N/Ts);
DeltaT=p*(2^N-1)*Ts*4;

[input_LUT_dSPACE,Tfin] = generate_input_signal(Ts,t1,DeltaT,N,p,u_star,delta,delta_spab);



%% Data acquisition (use t, u, y to perform system identification)
out = sim("GULIN_GhitaDaniel_convertor.slx");

t = out.tout;
u = out.u;
y = out.y;

subplot(211)
plot(t,u)
subplot(212)
plot(t,y)

%% System identification

i1=42431;
i2=135139;
i3=168876;
i4=246097;

Nr=20;

t_id=t(i1:Nr:i2);
u_id=u(i1:Nr:i2);
y_id=y(i1:Nr:i2);
t_vd=t(i3:Nr:i4);
u_vd=u(i3:Nr:i4);
y_vd=y(i3:Nr:i4);
```

```
u_id=u_id-mean(u_id);
y_id=y_id-mean(y_id);
u_vd=u_vd-mean(u_vd);
y_vd=y_vd-mean(y_vd);

figure
subplot(221)
plot(t_id,u_id)
subplot(223)
plot(t_id,y_id)
subplot(222)
plot(t_vd,u_vd)
subplot(224)
plot(t_vd,y_vd)
%%
dat_id = iddata(y_id,u_id,t_id(2)-t_id(1));
dat_vd=iddata(y_vd,u_vd,t_vd(2)-t_vd(1));
%%
model_arx=arx(dat_id,[6,6,1])
figure,resid(model_arx,dat_vd)
figure,compare(model_arx,dat_vd)
%%
model_armax=armax(dat_id,[4,4,2,1])
figure,resid(model_armax,dat_vd)
figure,compare(model_armax,dat_vd)
%%
model_oe=oe(dat_id,[2,2,1])
figure,resid(model_oe,dat_vd)
figure,compare(model_oe,dat_vd)
%%
model_n4sid = n4sid(dat_id,8)
figure,resid(model_n4sid,dat_vd)
figure,compare(model_n4sid,dat_vd)
%%
model_ssest = ssest(dat_id,4)
figure,resid(model_ssest,dat_vd)
figure,compare(model_ssest,dat_vd)
%%
pole(model_ssest)

zpk(model_ssest)
```